

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Curso: Programación Orientada a Objetos (100000I55N)

Ciclo 2026 – 1 | Marzo



**Universidad
Tecnológica
del Perú**

GUÍA Y RÚBRICA PROYECTO FINAL

Sistema con POO + MySQL

Mg. Pedro Hernan De La Cruz Velazco

Modalidad	Grupal (máximo 4 integrantes)
Semana de entrega	Semana 18 – Sesión 35
Ponderación	40% de la nota final del curso
Formato de entrega	Repositorio GitHub +plataforma UTP + Sustentación oral en clase
Nota mínima aprobatoria	12 / 20

1. INTRODUCCIÓN

El Proyecto Final representa la culminación del aprendizaje del curso de Programación Orientada a Objetos. Los estudiantes deberán aplicar de forma integrada todos los conceptos trabajados a lo largo del ciclo para desarrollar un sistema de software funcional, útil y persistente.

El sistema debe evidenciar:

- Diseño y programación orientada a objetos con los cuatro pilares del paradigma.
- Uso de patrones de diseño con clases abstractas, interfaces y herencia.
- Persistencia de datos mediante una base de datos relacional MySQL.
- Colecciones, clases genéricas y expresiones lambda/Streams.
- Utilidad práctica: debe resolver un problema real o simular un sistema del mundo real.

2. REQUISITOS OBLIGATORIOS DEL PROYECTO

2.1 Tema y Utilidad

El proyecto debe abordar un problema o necesidad real. Algunos ejemplos orientativos:

- Sistema de gestión de biblioteca o inventario.
- Sistema de reservas (hotel, clínica, restaurante, transporte).
- Plataforma de seguimiento académico o de empleados.
- Sistema de ventas o facturación.
- Aplicación de gestión de proyectos o tareas.

El tema debe ser propuesto por el grupo y aprobado por el docente en la Semana 2.

2.2 Requisitos de Programación Orientada a Objetos

El sistema DEBE implementar obligatoriamente:

1. Al menos 6 clases propias del dominio del problema.
2. Aplicación de los 4 pilares: abstracción, encapsulamiento, herencia y polimorfismo.
3. Al menos 1 clase abstracta con métodos abstractos sobrescritos.
4. Al menos 1 interface implementada en múltiples clases.
5. Relaciones entre clases: asociación, composición y/o agregación.
6. Diagrama de clases UML completo entregado en formato PDF o imagen.
7. Uso de colecciones: al menos List y uno de Set o Map.
8. Al menos 2 expresiones lambda o Streams con operaciones sobre colecciones (filter, map, sorted, etc.).

2.3 Requisitos de Persistencia – Base de Datos MySQL

La persistencia de datos es un requisito FUNDAMENTAL del proyecto:

9. Conexión funcional a una base de datos MySQL local o en la nube.
10. Script SQL de creación de la base de datos (.sql) incluido en el repositorio.
11. Mínimo 3 tablas con relaciones (claves foráneas).

12. Implementación completa de operaciones CRUD (Create, Read, Update, Delete) para al menos 2 entidades.
13. Las clases del dominio deben mapear directamente con las tablas de la base de datos.
14. Manejo básico de excepciones de conexión (SQLException).

Tecnologías permitidas para la conexión:

- JDBC puro (obligatorio conocer).
- Opcional: uso de patrones DAO (Data Access Object) para mejor organización.

2.4 Estructura del Repositorio GitHub

El repositorio debe organizarse de la siguiente manera:

mi-proyecto-poo/	
src/	→ Código fuente Java
sql/	→ Script de creación y datos iniciales (.sql)
diagramas/	→ Diagrama UML (imagen o PDF)
docs/	→ Informe técnico del proyecto
README.md	→ Descripción, instrucciones de instalación

3. CRONOGRAMA DE AVANCES Y REVISIONES

SEMANA	HITO	ENTREGABLE / ACTIVIDAD	OBSERVACIÓN
2	Formación de grupos	Registro de integrantes, elección del tema y propuesta inicial aprobada por el docente.	Aprobación obligatoria del docente.
9	Avance 30%	Presentación del diseño: diagrama UML preliminar, jerarquía de clases definida, conexión básica a MySQL.	Exposición por grupos (5 min c/u). Sin nota, solo retroalimentación.
14	Avance 60%	CRUD funcional con MySQL, clases abstractas/interfaces implementadas, al menos 1 lambda funcional.	Exposición por grupos (8 min c/u). Sin nota, solo retroalimentación.
18	ENTREGA FINAL	Sistema completo + Repositorio GitHub + Informe técnico + Sustentación oral en clase.	Evaluación con rúbrica. Nota 40% del promedio final.

4. INFORME TÉCNICO – ESTRUCTURA

El grupo debe entregar un informe técnico en formato PDF (mínimo 10 páginas) con la siguiente estructura:

15. Carátula: nombre del sistema, integrantes, carrera, ciclo y fecha.
16. Descripción del sistema: problemática que resuelve, usuarios objetivo y funcionalidades principales.

17. Diagrama de clases UML completo y explicado.
18. Descripción de la arquitectura OOP: cómo se aplica cada pilar.
19. Modelo de base de datos: diagrama entidad-relación y explicación de tablas.
20. Script SQL de creación de base de datos (incluido también en el repositorio).
21. Fragmentos de código clave comentados: clases abstractas, interfaces, lambdas, CRUD.
22. Manual de instalación y ejecución del proyecto.
23. Conclusiones del equipo sobre el aprendizaje logrado.
24. Bibliografía consultada.

5. GUÍA PARA LA SUSTENTACIÓN ORAL

5.1 Estructura de la Presentación (20 minutos por grupo)

TIEMPO	CONTENIDO	RESPONSABLE
0 – 3 min	Introducción: nombre del sistema, problema que resuelve y usuarios objetivo.	Integrante designado
3 – 8 min	Exposición técnica: diagrama UML, decisiones de diseño OOP, estructura de clases y base de datos.	Todos los integrantes
8 – 15 min	Demostración en vivo: ejecución del sistema con el flujo completo (CRUD, colecciones, lambda). Mostrar la base de datos actualizada.	Integrante designado
15 – 20 min	Preguntas del docente a todos los integrantes sobre el código, diseño y decisiones técnicas.	Todos los integrantes

5.2 Recomendaciones para la Sustentación

- Todos los integrantes deben conocer el código completo del sistema, no solo su parte.
- Tener el sistema ejecutándose antes de que comience la presentación.
- Contar con MySQL activo y con datos de prueba ya cargados.
- Preparar al menos 3 casos de uso para la demostración (crear, buscar y actualizar un registro).
- Usar diapositivas o material visual de apoyo para la parte técnica.
- Hablar con fluidez y claridad; evitar leer directamente del código.

RÚBRICA DE EVALUACIÓN

Proyecto Final – Programación Orientada a Objetos (100000I55N)

CRITERIO DE EVALUACIÓN	EXCELENTE (100%)	BUENO (75%)	REGULAR (50%)	DEFICIENTE (25%)	PTOS MÁX.
1. DISEÑO Y ARQUITECTURA OOP					
Aplicación de los 4 pilares POO (Abstracción, Encapsulamiento, Herencia, Polimorfismo)	Implementa correctamente los 4 pilares con diseño sólido y coherente. Se evidencia en múltiples clases y relaciones.	Implementa al menos 3 pilares correctamente con algunos errores menores de diseño.	Implementa 2 pilares de forma básica o con errores significativos de diseño.	Implementa menos de 2 pilares o están mal aplicados.	20
Uso de clases abstractas e interfaces	Define e implementa clases abstractas e interfaces de forma pertinente y con herencia múltiple correcta.	Usa clases abstractas o interfaces correctamente en al menos un módulo.	Intenta usar clases abstractas/interfaces pero con errores de implementación.	No usa clases abstractas ni interfaces.	
Diagrama de clases UML	Diagrama UML completo, correcto y consistente con el código. Muestra todas las relaciones (herencia, asociación, composición, interfaces).	Diagrama UML correcto con algunas relaciones faltantes o con errores menores.	Diagrama UML incompleto o con varios errores de notación.	No presenta diagrama UML o es irreconocible.	10
2. FUNCIONALIDAD Y UTILIDAD					
Utilidad práctica del sistema (problema real resuelto)	El sistema resuelve un problema real y concreto de forma completa, con CRUD funcional y flujo de usuario coherente.	El sistema resuelve un problema real con funcionalidad mayormente completa (>70%).	El sistema aborda un problema real pero con funcionalidad incompleta o con errores.	El sistema no tiene utilidad práctica clara o no funciona.	15
Uso de Colecciones (List, Set, Map)	Usa al menos 2 tipos de colecciones de forma apropiada y eficiente en el	Usa al menos 1 tipo de colección correctamente.	Usa colecciones pero de forma incorrecta o sin propósito claro.	No usa colecciones o las usa como simples arrays.	5

CRITERIO DE EVALUACIÓN	EXCELENTE (100%)	BUENO (75%)	REGULAR (50%)	DEFICIENTE (25%)	PTOS MÁX.
	flujo del sistema.				
Programación funcional / Lambda	Implementa expresiones lambda y Streams de forma correcta y en al menos 2 operaciones distintas (filtrar, mapear, ordenar, etc.).	Usa expresiones lambda en al menos 1 operación correcta.	Intenta usar lambda pero con errores de sintaxis o lógica.	No usa expresiones lambda ni Streams.	5
3. PERSISTENCIA DE DATOS – BASE DE DATOS MySQL					
Conexión y configuración de MySQL	Conexión robusta con manejo de errores, credenciales externalizadas y base de datos propia del proyecto.	Conexión funcional sin manejo de errores o con credenciales hardcoded.	Conexión parcialmente funcional o con errores frecuentes.	No hay conexión a MySQL o no funciona.	10
Operaciones CRUD con MySQL (Crear, Leer, Actualizar, Eliminar)	Las 4 operaciones CRUD funcionan correctamente y están integradas al flujo de la aplicación.	Al menos 3 operaciones CRUD funcionan correctamente.	Al menos 2 operaciones CRUD funcionan correctamente.	Menos de 2 operaciones CRUD funcionan.	10
Diseño del esquema de base de datos (tablas, relaciones, integridad)	Esquema normalizado (mínimo 3 tablas), con claves foráneas, relaciones correctas e índices donde corresponde.	Esquema con mínimo 2 tablas con relación correcta pero sin normalización completa.	Esquema básico con 1-2 tablas sin relaciones entre ellas.	Esquema inexistente o mal diseñado.	5
Clases persistentes / Record Classes / Sealed Classes	Implementa clases persistentes con serialización o usa Record/Sealed Classes de forma correcta y apropiada.	Implementa clases persistentes básicas sin uso de Record/Sealed Classes.	Intenta implementar persistencia pero con errores de diseño.	No implementa ningún concepto de clases persistentes.	5
4. EXPOSICIÓN Y SUSTENTACIÓN					

CRITERIO DE EVALUACIÓN	EXCELENTE (100%)	BUENO (75%)	REGULAR (50%)	DEFICIENTE (25%)	PTOS MÁX.
Dominio del tema y respuesta a preguntas	Todos los integrantes demuestran dominio técnico profundo y responden correctamente a las preguntas del docente.	La mayoría responde correctamente, con algunas respuestas incompletas.	Solo uno o dos integrantes pueden responder las preguntas.	El grupo no puede responder las preguntas técnicas.	10
Claridad de la presentación y demostración en vivo	Presentación fluida, bien organizada, con demostración del sistema ejecutándose en vivo sin errores.	Presentación clara con demostración en vivo con errores menores.	Presentación desorganizada o demostración incompleta.	No hay demostración funcional del sistema.	5
PUNTAJE TOTAL					100

Escala de Conversión de Puntaje a Nota

90 – 100 pts	80 – 89 pts	70 – 79 pts	60 – 69 pts	50 – 59 pts	< 50 pts
Nota: 18 – 20	Nota: 16 – 17	Nota: 14 – 15	Nota: 12 – 13	Nota: 10 – 11	Nota: < 10

6. PENALIZACIONES Y CONSIDERACIONES ÉTICAS

SITUACIÓN	PENALIZACIÓN
Entrega del repositorio GitHub después de la fecha límite (hasta 48 hrs de retraso).	–2 puntos por día de retraso.
Integrante que no puede responder ninguna pregunta técnica durante la sustentación.	–5 puntos al integrante.
Código copiado de otro grupo o de internet sin adaptación ni comprensión demostrada.	Nota 00 al grupo y reporte a coordinación académica.
Uso de código generado íntegramente por IA sin comprensión (detectado en sustentación).	–10 puntos al grupo.
No presentar el diagrama UML.	–10 puntos del criterio correspondiente.
Sistema no conecta a MySQL o no tiene base de datos funcional.	Máximo 50 puntos en la categoría de persistencia.

7. HOJA DE EVALUACIÓN – USO EXCLUSIVO DEL DOCENTE

Nombre del grupo / Sistema:	
Integrantes:	1. 2. 3. 4.
Fecha de sustentación:	
Repositorio GitHub:	

CRITERIO	Exc.	Bue.	Reg.	PUNTAJE
1. DISEÑO Y ARQUITECTURA OOP				
Aplicación de los 4 pilares POO — (Abstracción, Encapsulamiento, Herencia, Polimorfismo) [20 pts]	☐	☐	☐	—
Uso de clases abstractas e interfaces [10 pts]	☐	☐	☐	—
Diagrama de clases UML [10 pts]	☐	☐	☐	—
2. FUNCIONALIDAD Y UTILIDAD				
Utilidad práctica del sistema — (problema real resuelto) [15 pts]	☐	☐	☐	—
Uso de Colecciones (List, Set, Map) [5 pts]	☐	☐	☐	—
Programación funcional / Lambda [5 pts]	☐	☐	☐	—
3. PERSISTENCIA DE DATOS – BASE DE DATOS MySQL				
Conexión y configuración de MySQL [10 pts]	☐	☐	☐	—
Operaciones CRUD con MySQL — (Crear, Leer, Actualizar, Eliminar) [10 pts]	☐	☐	☐	—
Diseño del esquema de base de datos — (tablas, relaciones, integridad) [5 pts]	☐	☐	☐	—
Clases persistentes / Record Classes / Sealed Classes [5 pts]	☐	☐	☐	—
4. EXPOSICIÓN Y SUSTENTACIÓN				
Dominio del tema y respuesta a preguntas [10 pts]	☐	☐	☐	—
Claridad de la presentación y demostración en vivo [5 pts]	☐	☐	☐	—
PUNTAJE TOTAL (sobre 100)				—
NOTA FINAL (escala 0–20)				—

Observaciones del docente:

Pedro Hernán De la Cruz Velazco
Firma del Docente